

# ME315A\_LAB05

Letícia Santos Tavares

## Laboratório Especial

### Parte 1

```
#chamando todas as bibliotecas que serão utilizadas
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(ggplot2)
library(leaflet)
library(ggcal)
library(readr)
```

Crie uma função que:

- Receba um valor de TAIL\_NUMBER (por exemplo, N431WN);
- Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip);

- c. Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima);
- d. O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do voo como espessura da linha que conecta os aeroportos envolvidos no trajeto);

```
#CRIANDO A FUNÇÃO
analisa_aeronave <- function(tail_number, arquivo) {

  # Lê o arquivo flights em pedaços de 10.000 linhas
  voos <- read_csv_chunked(
    arquivo,
    callback = DataFrameCallback$new(function(chunk, pos) {

      # Mantém somente os voos da aeronave escolhida
      chunk |>
        filter(TAIL_NUMBER == tail_number)

    })),
    chunk_size = 10000
  )

  # Ordena os voos pela data e pelo horário
  voos <- voos |>
    arrange(YEAR, MONTH, DAY, SCHEDULED_DEPARTURE)

  # Importa as informações dos aeroportos
  airports <- read_csv(
    "airports.csv",
    show_col_types = FALSE
  )

  # Seleciona as coordenadas dos aeroportos
  coordenadas <- airports |>
    select(IATA_CODE, LATITUDE, LONGITUDE)

  # Adiciona as coordenadas do aeroporto de origem
  voos <- voos |>
    left_join(
      coordenadas,
      by = c("ORIGIN_AIRPORT" = "IATA_CODE")
    )
}
```

```

# Adiciona as coordenadas do aeroporto de destino
voos <- voos |>
  left_join(
    coordenadas,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE"),
    suffix = c("_ORIGIN", "_DESTINATION")
  )

# Calcula a velocidade média de cada voo
voos <- voos |>
  mutate(
    VELOCIDADE_MEDIA = DISTANCE / (AIR_TIME / 60)
  )

# Define a espessura das linhas com base na velocidade
# A espessura fica entre 1 e 4
velocidade_min <- min(voos$VELOCIDADE_MEDIA, na.rm = TRUE)
velocidade_max <- max(voos$VELOCIDADE_MEDIA, na.rm = TRUE)

voos <- voos |>
  mutate(
    ESPESSURA = 1 + 3 *
      (VELOCIDADE_MEDIA - velocidade_min) /
      (velocidade_max - velocidade_min)
  )

# Cria o mapa
mapa <- leaflet() |>
  addTiles()

# Adiciona cada trajeto ao mapa
for (i in 1:nrow(voos)) {

  # Só desenha o trajeto se houver coordenadas e velocidade
  if (
    !is.na(voos$LONGITUDE_ORIGIN[i]) &&
    !is.na(voos$LONGITUDE_DESTINATION[i]) &&
    !is.na(voos$LATITUDE_ORIGIN[i]) &&
    !is.na(voos$LATITUDE_DESTINATION[i]) &&
    !is.na(voos$ESPESSURA[i])
  ) {

```

```

# Informações mostradas ao clicar no trajeto
informacoes <- paste0(
  "<b>Origem:</b> ", voos$ORIGIN_AIRPORT[i],
  "<br><b>Destino:</b> ", voos$DESTINATION_AIRPORT[i],
  "<br><b>Data:</b> ",
  voos$YEAR[i], "-", voos$MONTH[i], "-", voos$DAY[i],
  "<br><b>Voo:</b> ", voos$FLIGHT_NUMBER[i],
  "<br><b>Velocidade média:</b> ",
  round(voos$VELOCIDADE_MEDIA[i], 1)
)

mapa <- mapa |>
  addPolylines(
    lng = c(
      voos$LONGITUDE_ORIGIN[i],
      voos$LONGITUDE_DESTINATION[i]
    ),
    lat = c(
      voos$LATITUDE_ORIGIN[i],
      voos$LATITUDE_DESTINATION[i]
    ),
    weight = voos$ESPESSURA[i],
    opacity = 0.6,
    popup = informacoes
  )
}
}

# Cria uma tabela com os aeroportos utilizados
aeroportos <- bind_rows(
  voos |>
    select(
      codigo = ORIGIN_AIRPORT,
      latitude = LATITUDE_ORIGIN,
      longitude = LONGITUDE_ORIGIN
    ),
  voos |>
    select(
      codigo = DESTINATION_AIRPORT,
      latitude = LATITUDE_DESTINATION,
      longitude = LONGITUDE_DESTINATION
    )
)

```

```

) |>
  distinct(codigo, .keep_all = TRUE)

# Adiciona os aeroportos ao mapa
mapa <- mapa |>
  addCircleMarkers(
    data = aeroportos,
    lng = ~longitude,
    lat = ~latitude,
    radius = 5,
    popup = ~paste0("<b>Aeroporto:</b> ", codigo)
  )

# Retorna a tabela e o mapa
return(
  list(
    tabela = voos,
    grafico = mapa
  )
)
}

```

```

#TESTANDO A FUNÇÃO PARA UMA AERONAVE
analisa_aeronave('N3HYAA','flights.csv')

```

```

Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()

```

```
[1] "2026-09-15 12:59:09 -03"
```